

Project III: Network Algorithms

Will Clark

June 4, 2026

Abstract

This report analyzes structural properties of Barabasi-Albert randomly generated networks. It explores diameter, clustering, and the degree distribution benchmarked on empirical data/results.

1 Introduction

Scientists from all fields are interacting and collaborating to understand the importance of, and structure behind, graphs and networks. Graphs enable us to model social interactions, design complex distributed networks, and model many biological interactions. In short, Networks are *everywhere*.

1.1 Structure

We aren't just concerned with graphs as a *construct*, but as a medium to exploit certain structures. These structures, *topologies*, reveal communities, "important" vertices, and many more aspects. This report explores three structures in particular, namely the

- ◇ Degree Distribution
- ◇ Clustering Coefficient
- ◇ and Diameter

of various networks. Each of these structures highlight important variations in meaning, and different graph networks exhibit each structure differently. We use often these structures to characterize certain networks. The canonical example is derived from Travers and Milgram's *An Experimental Study of the Small World Problem*. By instructing participants in Nebraska to send letters to a stockbroker in Boston by *only* sending letters to friends, they showed that the average number of hops for successful letters was between 5 and 6. Formulated as a graph problem, we say the small world network as a small "diameter." As such, the ability to classify graphs based on their structure has many real-world applications, and is the backbone of analysis in many fields.

1.2 Barabasi-Albert

Gathering graph's is not as trivial as you may think, and as it turns out, being able to *randomly* generate network structures to simulate real-world phenomena is of paramount importance. As such, this report examines one way of generating these graphs: **Barabasi-Albert**.

1.2.1 Pseudocode

Algorithm 1 Barabási-Albert (Preferential Attachment)

```

1: procedure PREFERENTIAL ATTACHMENT( $n, d$ )
2:    $G = (\{0, \dots, n-1\}, E)$ 
3:    $M \leftarrow$  array of length  $2 \cdot n \cdot d$ 
4:   for  $v \leftarrow 1$  to  $n-1$  do
5:     for  $i \leftarrow 0$  to  $d-1$  do
6:        $M[2(v \cdot d + i)] \leftarrow v$ 
7:       Draw  $r \in \{0, \dots, 2(v \cdot d + i)\}$  uniformly at random
8:        $M[2(v \cdot d + i) + 1] \leftarrow M[r]$ 
9:     end for
10:  end for
11:   $E \leftarrow \emptyset$ 
12:  for  $i \leftarrow 0$  to  $n \cdot d - 1$  do
13:     $E \leftarrow E \cup \{M[2i], M[2i + 1]\}$ 
14:  end for
15:  return  $G$ 
16: end procedure

```

This algorithm is also called "preferential attachment," since we "prefer" earlier vertices instead of later ones. Formally, the probability p_i that a new node is connected to node i is

$$p_i = \frac{d_i}{\sum_j d_j} \quad d_k \text{ is the degree of the } k^{\text{th}} \text{ node}$$

And the denominator grows as $|V| \rightarrow N$, for $N \gg n > 0$. We are guaranteed a degeneracy of d , since by induction, if we pop the last vertex added, it's connected to d other vertices. Continuing this down, we see that the graph's degeneracy is determined precisely by d . Preferential attachment produces a *power law* distribution, where the probability $\mathbb{P}$ over a quantity X is

$$\mathbb{P}\{X\} \propto \frac{1}{X^\beta} \quad \beta \in \mathbb{R}$$

2 Degree Distributions

2.1 Definition

The degree distribution $\mathbb{P}(d)$ of a network $G = (V, E)$ is the fraction of nodes with degree d in the network.

$$\mathbb{P}(d) = \frac{|\{v \in G.V \mid \deg(v) = d\}|}{n} = \frac{n_d}{n}$$

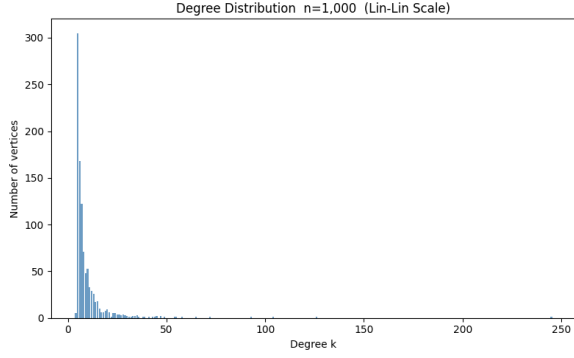
The degree distribution is incredibly important when studying real and theoretical networks. Most networks in the real world are *right skewed*, and some are conjectured to follow a power law. In that case, we say the network is *scale free*, since restricting the network to a subset of its nodes is statistically similar to the network in its totality.

2.2 Psuedocode

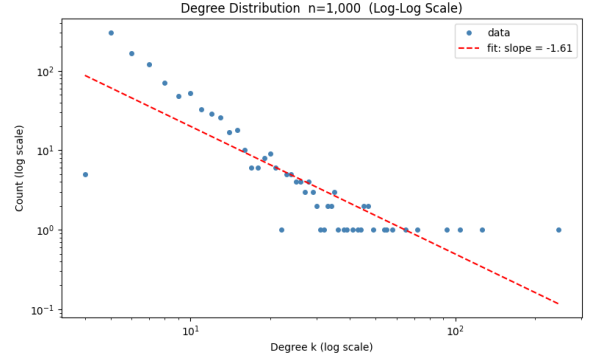
Algorithm 2 Degree Distribution

```
1: procedure DEGREEDISTRIBUTION( $G$ )
2:    $H = [0, 0, \dots, 0]$ ,    $|H| = n$ 
3:   for all  $v \in G.V$  do
4:      $d = \deg(v)$ 
5:      $H[d] + = 1$ 
6:   end for
7:   return  $H$ 
8: end procedure
```

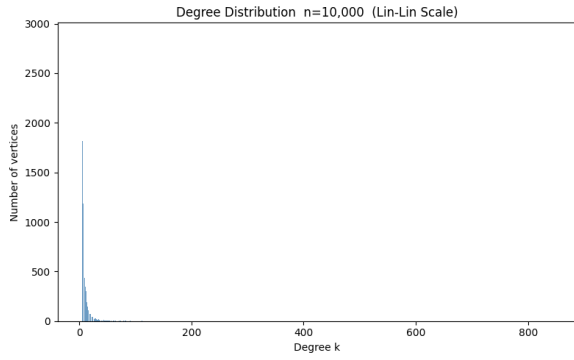
2.3 Analysis



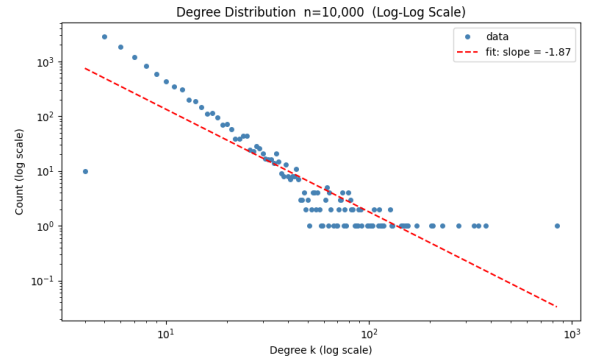
(a) $n = 1000$ (Left)



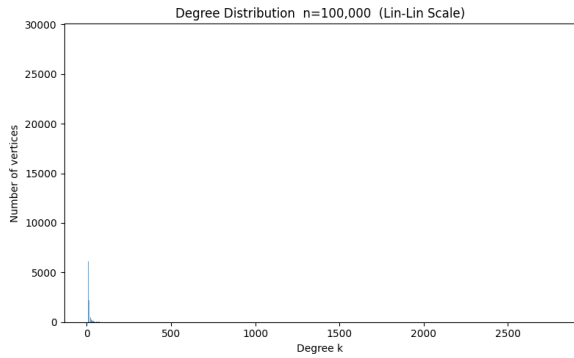
(b) $n = 1000$ (Right)



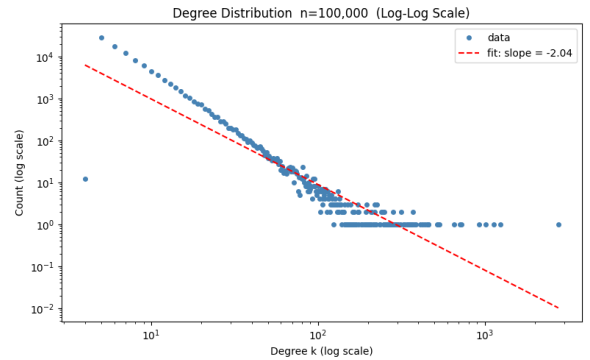
(c) $n = 10000$ (Left)



(d) $n = 10000$ (Right)



(e) $n = 100000$ (Left)



(f) $n = 100000$ (Right)

Figure 1: Degree distributions across different network sizes ($n = 1000, 10000, 100000$).

The degree distribution was analyzed for $n = 1000$, $n = 10000$, and $n = 100000$ vertices. On the left, "(Left)", of the figure, each distribution is plotted on a lin-lin scale. We can see the **heavy right skew** characteristic of a power law, but it's virtually impossible to quantify the exponent β is. Transforming the data to a Log-Log scale exposes a clear linear trend, confirming *scale free* behavior. We fit a slope of -1.61 , -1.87 , -2.04 for each n value respectively. Consequently, as $n \rightarrow N$, the degree distributions converge towards steeper power-law decay. As such, the *Barabasi-Albert* random graphs follow a **power law**.

3 Clustering Coefficients

3.1 Definition

The clustering coefficient is

$$C = \frac{3 \cdot (\text{number of triangles})}{\text{number of 2-edge paths}}.$$

The denominator is cheap; the numerator is computed via a degeneracy ordering so that triangle counting runs in $O(d^2n) = O(dm)$ expected time rather than the naïve $O(n^3)$.

3.2 Psuedocode

Algorithm 3 Number of 2-Edge Paths

```

1: procedure NUM2EDGEPATHS( $G$ )
2:   total  $\leftarrow 0$ 
3:   for all  $v \in G.V$  do
4:      $d \leftarrow \deg(v)$ 
5:     total  $\leftarrow$  total +  $\frac{d(d-1)}{2}$  ▷ paths with  $v$  in the middle
6:   end for
7:   return total
8: end procedure

```

Algorithm 4 Degeneracy Ordering

```

1: procedure DEGENERACYORDERING( $G$ )
2:    $L \leftarrow []$ ,  $H_L \leftarrow \emptyset$  ▷ output ordering / vertices placed in  $L$ 
3:   for all  $v \in G.V$  do
4:      $d_v \leftarrow \deg(v)$ ,  $N_v \leftarrow []$  ▷  $N_v$ : neighbors of  $v$  before  $v$  in  $L$ 
5:   end for
6:   for all  $v \in G.V$  do
7:      $D[d_v] \leftarrow D[d_v] \cup \{v\}$  ▷ bucket vertices by current degree
8:   end for
9:    $k \leftarrow 0$ 
10:  for  $j \leftarrow 1$  to  $n$  do
11:     $i \leftarrow \min\{i : D[i] \neq \emptyset\}$ 
12:     $k \leftarrow \max(k, i)$ 
13:     $v \leftarrow$  any vertex in  $D[i]$ 
14:     $D[i] \leftarrow D[i] \setminus \{v\}$ 
15:    prepend  $v$  to  $L$ ;  $H_L \leftarrow H_L \cup \{v\}$ 
16:    for all  $w \in G.NEIGHBORS(v)$  with  $w \notin H_L$  do
17:       $D[d_w] \leftarrow D[d_w] \setminus \{w\}$ 
18:       $d_w \leftarrow d_w - 1$ 
19:       $D[d_w] \leftarrow D[d_w] \cup \{w\}$ 
20:      append  $w$  to  $N_v$ 
21:    end for
22:  end for
23:  return ( $L, N$ )
24: end procedure

```

Algorithm 5 Counting Triangles

```
1: procedure COUNTTRIANGLES( $G$ )
2:    $(L, N) \leftarrow \text{DEGENERACYORDERING}(G)$ 
3:    $\Delta \leftarrow 0$ 
4:   for all  $v \in L$  do
5:     for all unordered pairs  $\{u, w\} \subseteq N_v$  do
6:       if  $(u, w) \in E$  then  $\triangleright O(1)$  adjacency lookup
7:          $\Delta \leftarrow \Delta + 1$ 
8:       end if
9:     end for
10:  end for
11:  return  $\Delta$ 
12: end procedure
```

Algorithm 6 Clustering Coefficient

```
1: procedure CLUSTERINGCOEFFICIENT( $G$ )
2:    $\Delta \leftarrow \text{COUNTTRIANGLES}(G)$ 
3:    $P \leftarrow \text{NUM2EDGEPATHS}(G)$ 
4:   return  $\frac{3\Delta}{P}$ 
5: end procedure
```

3.3 Analysis

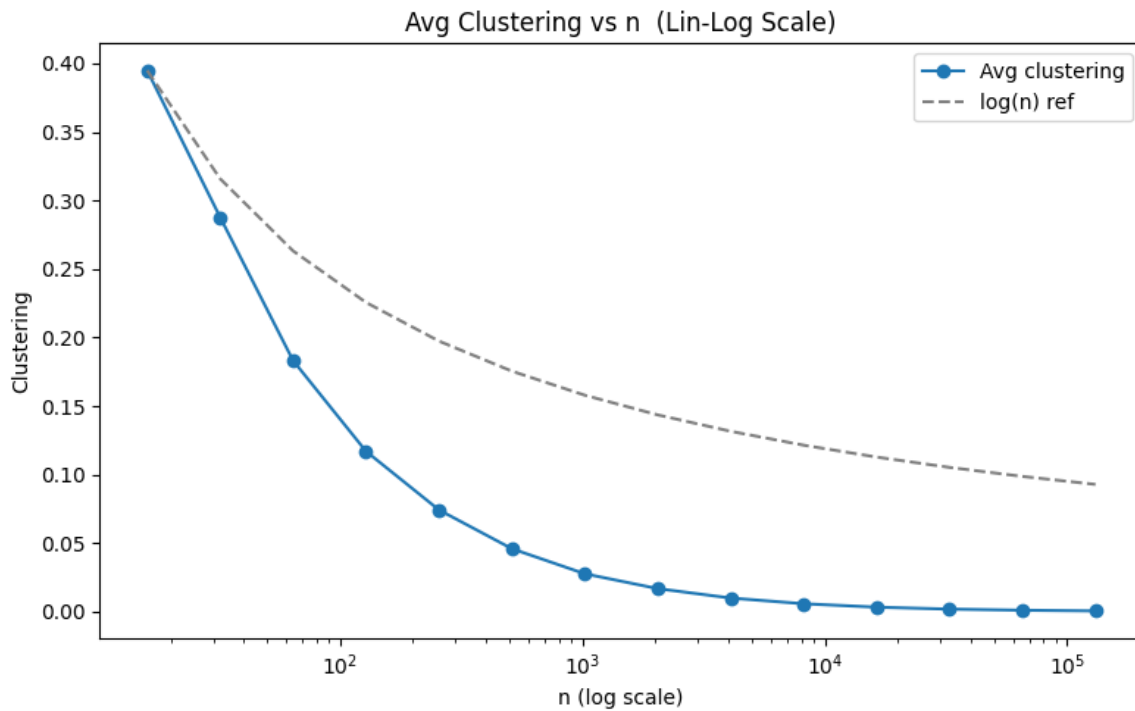


Figure 2: Average clustering vs n

The clustering coefficient, $\mathcal{C}$, measures how "cliquey" the network is. As shown in 2, as n increases, $\mathcal{C}$ decreases. This makes sense, since as we add more vertices via BA [1], the *still* form connections to *only* d other vertices, but the denominator scales with n .

I plotted a $\frac{1}{\log_2(n)}$ reference (the legend is incorrect), which suggests that

$$\mathcal{C} \propto \frac{1}{2^x}$$

Since the x axis maps the metric logarithmically, we can isolate the scaling function relative to the visual coordinates by substituting $n = 2^x$. Consequently, the exponential term in the denominator dominates the numerator, and $\mathcal{C}$ mimics pure exponential decay.

4 Diameters

4.1 Definition

The diameter, d , is the greatest distance between any pair of vertices, namely

$$d = \max_{v \in G.V} \max_{u \in G.V} d(v, u)$$

where $d: G.V \times G.V \rightarrow \mathbb{R}^+ \cup \{0\}$.

4.2 Psuedocode

Algorithm 7 Diameter

```

1: procedure HUERISTIC II( $G$ )
2:    $r \leftarrow \mathcal{U}(1, n)$  ▷ A random vertex
3:    $D_{\max} \leftarrow 0$ 
4:   while True do
5:      $(w, \text{dist}) \leftarrow \text{BFS}(\text{Graph}, r)$ 
6:     if  $\text{dist} > D_{\max}$  then
7:        $D_{\max} \leftarrow \text{dist}$ 
8:        $r \leftarrow w$ 
9:     else
10:      break
11:    end if
12:  end while
13:  return  $D_{\max}$ 
14: end procedure

```

4.3 Analysis

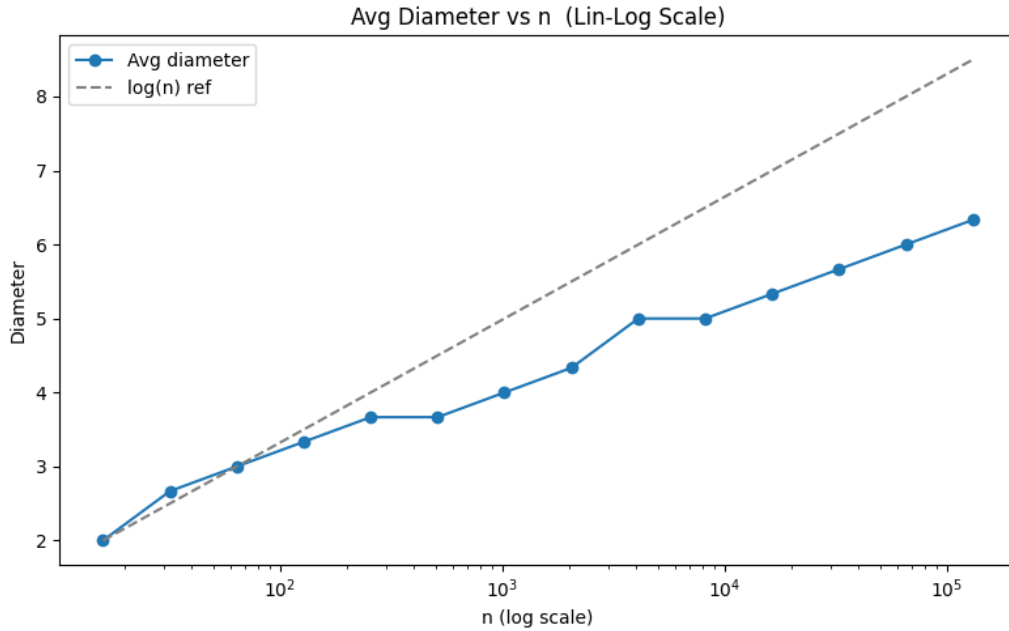


Figure 3: Average Diameter vs n

As shown in Figure 3, the average diameter increases with n , roughly tracking the $\log(n)$ reference line on the lin-log scale. This confirms $d \sim \Theta(\log n)$: preferential attachment produces high-degree hubs that act as shortcuts, keeping pairwise distances short even as the graph grows. The jagged effect visible at certain values of n is an artifact of diameter being integer-valued, since the average must accumulate enough samples before incrementing to the next integer. As such, the diameter **is changing as a function of n** .